

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Домашняя работа №5

Выполнила:

Крапивина Яна

Группа
J3212

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках данной работы нужно изучить основные команды пакетного менеджера NPM и научиться стартовать проект на Vue.

Ход работы

Работа состояла из двух этапов: развертывания рабочего окружения через терминал и написания кода с использованием архитектуры Vue.js. За основу для переноса на фреймворк был взят ранее разработанный функционал страницы «Транзакции» из проекта MyFinance.

- Работа с NPM и инициализация проекта

1. Создание проекта:

В рабочей директории была выполнена команда инициализации современного Vue-приложения:

```
npm init vue@latest
```

От установки дополнительных модулей (TypeScript, Router, Pinia) я отказалась, чтобы установить их вручную согласно мануалу

2. Установка зависимостей:

После перехода в папку проекта были скачаны базовые пакеты командой `npm install`.

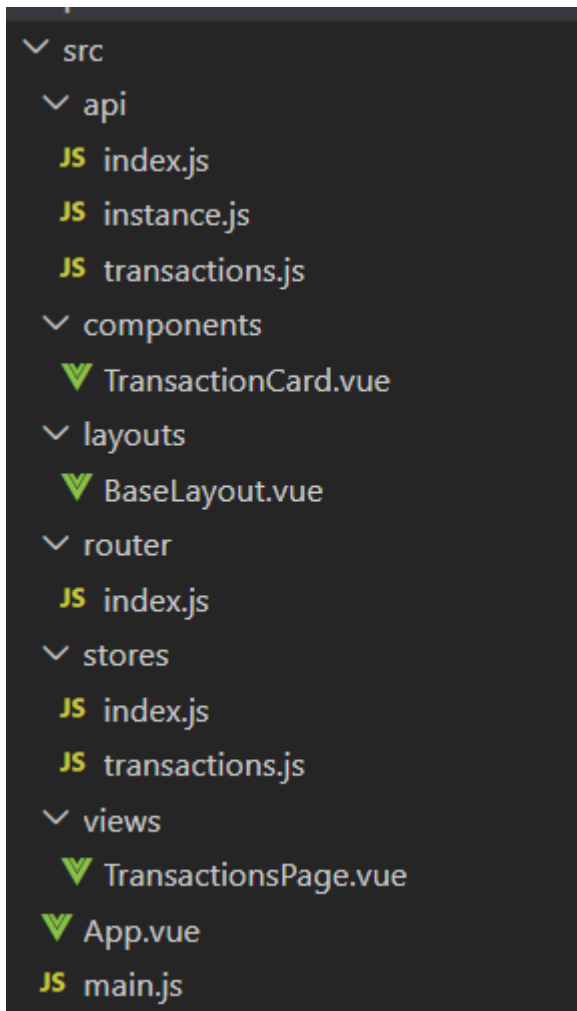
Далее, для реализации архитектуры приложения, одной командой были установлены необходимые библиотеки с флагом `-S` (сохранение в `package.json`):

```
npm i axios pinia-persists bootstrap vue-router pinia -S
```

- Vue Router — для маршрутизации (SPA)
- Pinia — для глобального стейт-менеджмента
- Axios — для HTTP-запросов к моковому API (json-server)
- Bootstrap — для стилизации компонентов

- Архитектура приложения на Vue.js

После генерации проекта из папки `src` были удалены демонстрационные компоненты. Вместо них была воссоздана строгая модульная структура, описанная в мануале.



Логика приложения была разделена на следующие слои:

Слой API (src/api):

В файле `instance.js` настроен базовый URL для `axios` (указывающий на локальный `json-server` из ЛР). В файле `transactions.js` создан класс `TransactionsApi` с методами `getAll` (получение списка) и `createTransaction` (POST-запрос на добавление). Для инкапсуляции применен принцип `Dependency Injection` (инстанс `axios` передается в конструктор).

Стейт-менеджмент (src/stores):

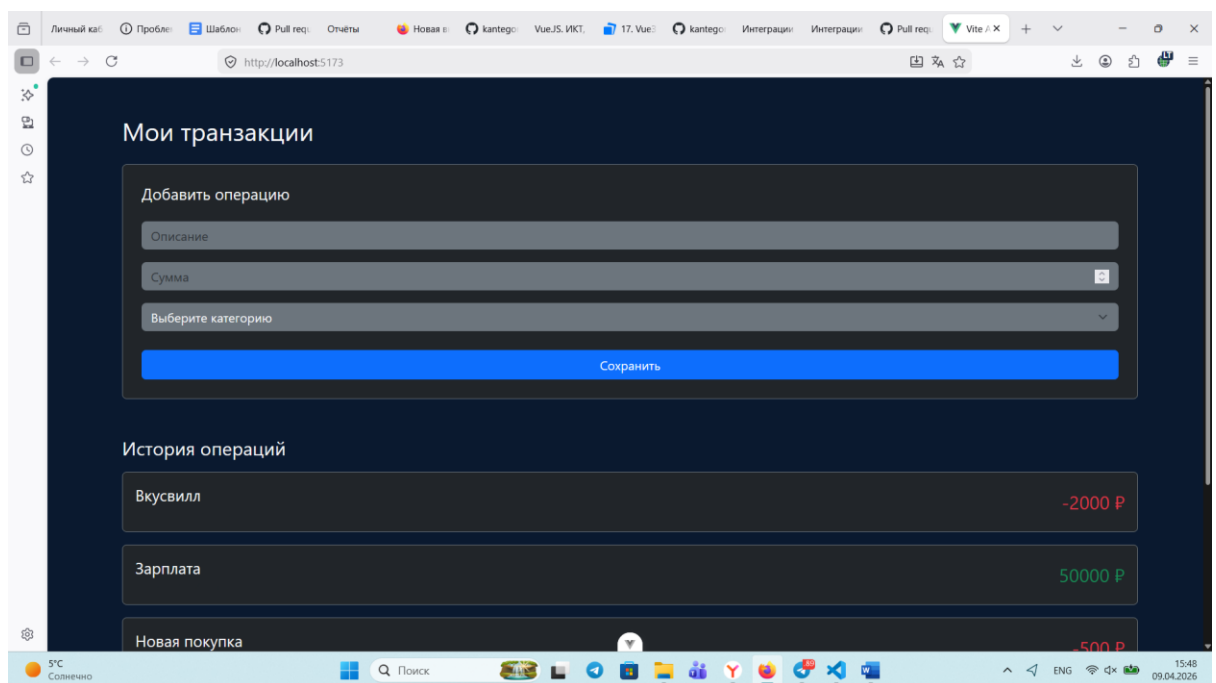
Подключена библиотека `Pinia`. В файле `transactions.js` создано глобальное хранилище. Оно содержит массив транзакций и `actions` асинхронные методы `loadTransactions` и `createTransaction`, которые обращаются к слою API и обновляют состояние.

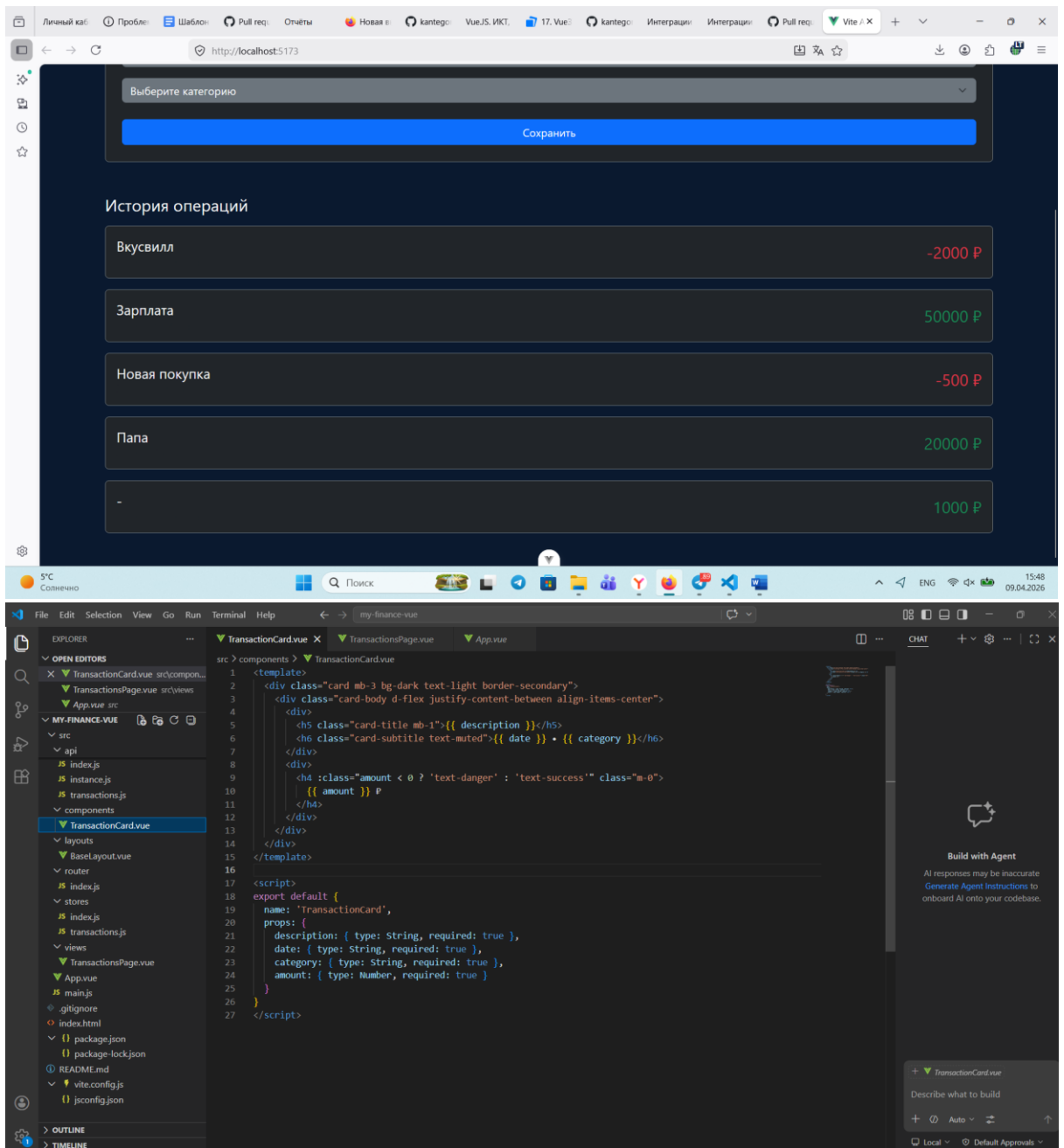
Роутинг (src/router):

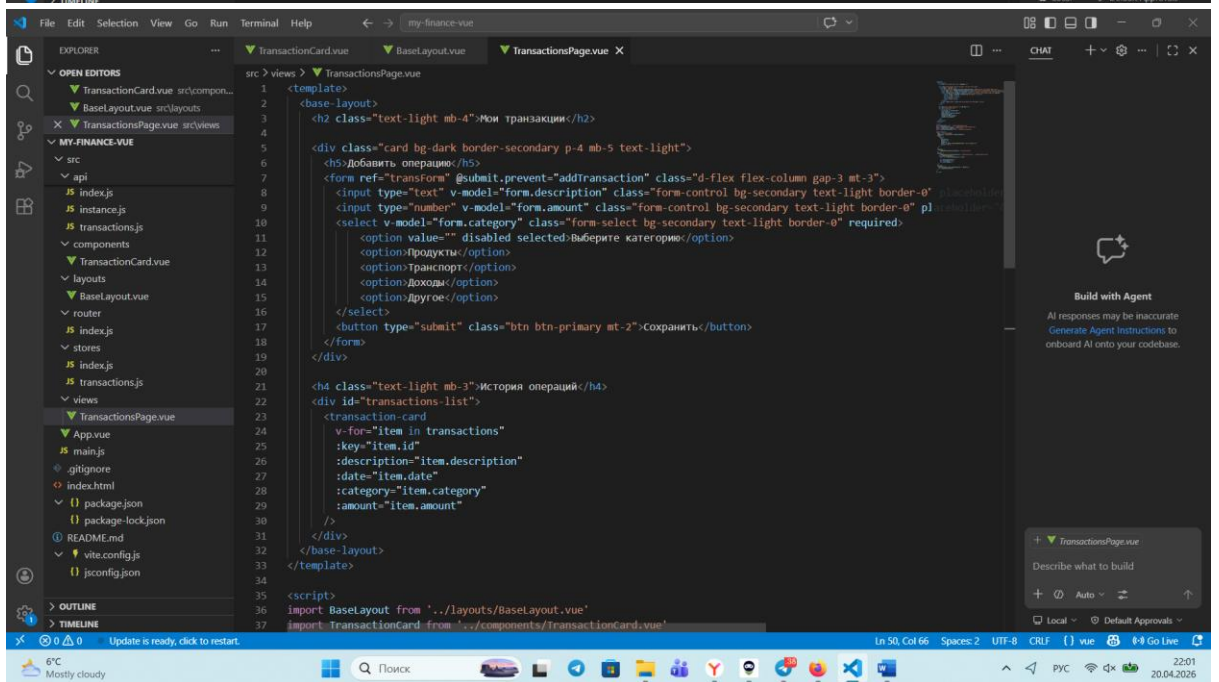
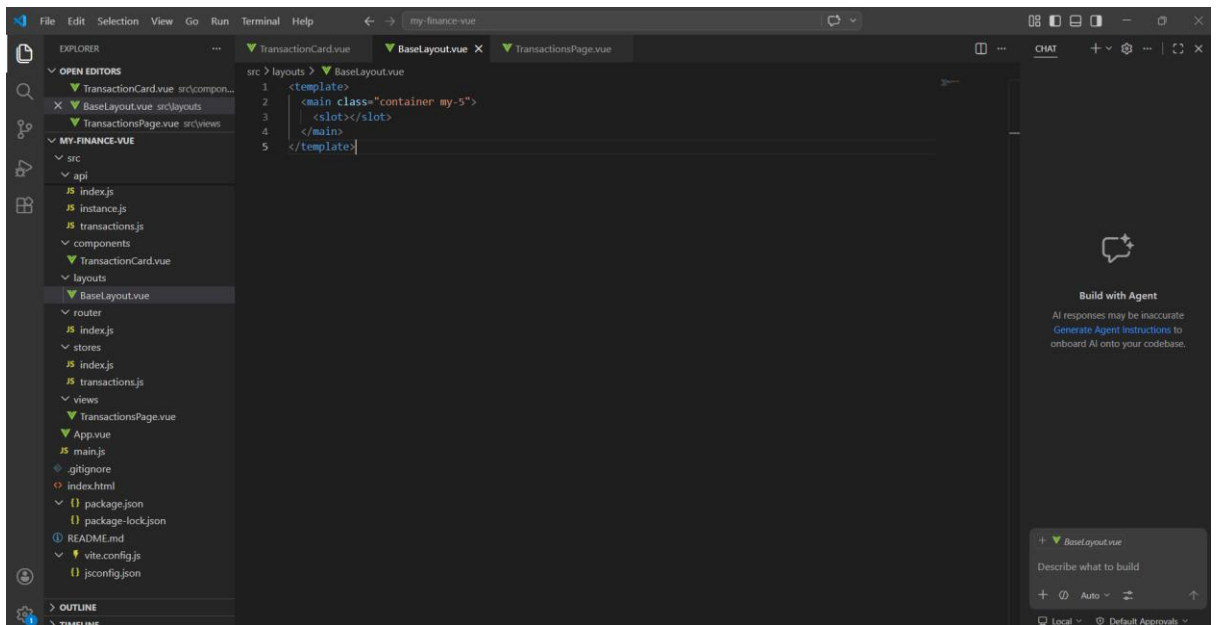
В index.js настроен vue-router с ленивой загрузкой. Создан единственный маршрут, который рендерит страницу TransactionsPage.vue.

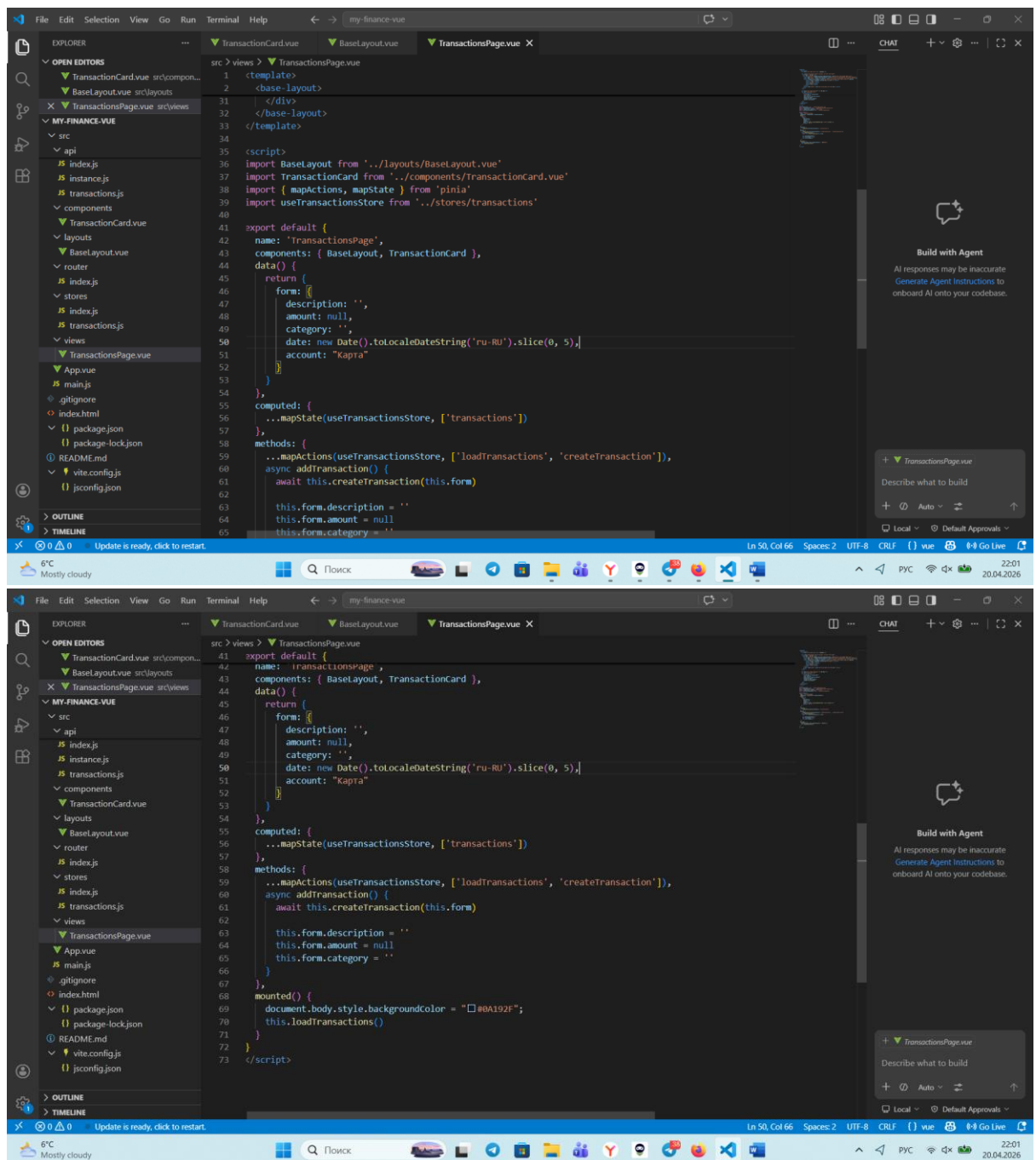
Слой представления (Views, Components, Layouts):

- BaseLayout.vue: Компонент-обертка, задающий основные отступы контейнера через <slot />
- TransactionCard.vue: UI-компонент карточки транзакции. Принимает данные через props (описание, дата, категория, сумма) и отрисовывает их
- TransactionsPage.vue: Главная страница









Вывод

В ходе выполнения домашней работы были успешно освоены базовые принципы работы с пакетным менеджером NPM и фреймворком Vue 3.

Приложение было инициализировано с нуля через CLI и очищено от шаблонного кода. Разработана масштабируемая архитектура, четко разделяющая логику запросов (Axios), глобальное состояние (Pinia),

маршрутизацию (Vue Router) и пользовательский интерфейс (переиспользуемые Vue-компоненты).